Section A:

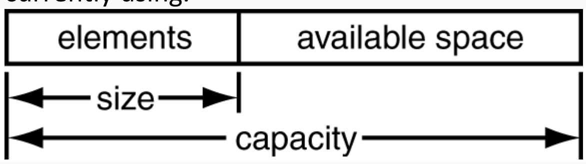
1. Take your code from lab 3B where you developed your own vector class MyIntVector (You will have 2 files MyIntVector.h which contains the declaration and MyIntVector.cpp which contains the implementation). Do not use the build in vector <vector> and add the following method to your MyIntVector

   Add only the public methods below: (more and you get 0 marks)

   - MyIntVector (const MyIntVector &t) a user defined Copy Constructor that makes a deep copy (includes copying the pointed to data) of you MyIntVector
   - friend ostream& operator<<(ostream &str, const MyIntVector &myVec); an overloaded << that outputs the MyIntVector to the screen.

Remember your MyIntVector must only have the 3 private attributes: (more and you get 0 marks)

- m_size: the number of items currently in the vector
- m_capacity: Capacity is the amount of allocated storage capacity that the vector is currently using.



- arrPtr: A pointer to the first element of the underlying array

Section B

Write a main program (source.cpp) to test the MyIntVector copy constructor and overloaded << that you have made in section A. Create an object of MyIntVector in the source file, then make a copy of that object and then output both objects to the screen using your << operator to test that they are the same.

Then try to have the compiler call the copy constructor implicitly. Remember in C++, a Copy Constructor may be called in the following cases:

- When an object of the class is returned by value.
- When an object of the class is passed (to a function) by value as an argument.
- When an object is constructed based on another object of the same class.
- When the compiler generates a temporary object.

For more info on copy constructors look here: https://www.geeksforgeeks.org/copy-constructor-in-cpp/

For more info on friend functions look here: https://www.geeksforgeeks.org/friend-class-function-cpp/